ITS Villada

Base de Datos II

TRABAJO PRÁCTICO INTEGRADOR ů GRUPO 8

Procesamiento distribuido y *machine learning* sobre datos masivos con Apache Spark MLlib

K-Means distribuido como actividad en el aula

Integrantes

Leon Guchea
Bautista Daveloze
Riccioni Santiago
Sibona Luciano
Macias Luciano

Docentes

Lisandro Márquez
Teodoro Reyna

9 de noviembre de 2026

Procesamiento distribuido y *machine learning* sobre datos masivos con Apache Spark MLlib: K-Means distribuido como actividad en el aula

Grupo 8
ITS Villada
Base de Datos II ů Trabajo Práctico Integrador
Integrantes: Leon Guchea, Bautista Daveloze, Riccioni Santiago,
Sibona Luciano y Macias Luciano
Docentes: Lisandro Márquez y Teodoro Reyna
Fecha: 9 de noviembre de 2026

***Resumen*—Cuando un conjunto de datos supera la capacidad de una sola computadora, su análisis exige repartir datos y cómputo entre muchas máquinas. Este informe presenta los fundamentos del procesamiento distribuido, desde MapReduce hasta Apache Spark, sus RDD y DataFrames, y describe cómo MLlib ejecuta K-Means sobre un clúster. Se analizan la agregación de sumas y conteos, la persistencia en memoria y la recuperación por linaje. Además, se comparan los usos de Spark con los de una base de datos relacional y se presenta una experiencia presencial que contrasta una mesa que procesa el conjunto completo con cuatro mesas que procesan particiones, seguida de una simulación web de K-Means.**

***Palabras clave*—Apache Spark, MLlib, MapReduce, Hadoop, K-Means, procesamiento distribuido, RDD, tolerancia a fallos, big data.**

I. Introducción

El volumen de datos que generan hoy las organizaciones (registros de transacciones, eventos de aplicaciones, sensores) crece más rápido que la capacidad de una única máquina para almacenarlos y procesarlos. Ante este límite hay dos caminos: escalar verticalmente, comprando una computadora más potente, o escalar horizontalmente, repartiendo el trabajo entre muchas computadoras comunes conectadas en red. El segundo camino es más barato y casi no tiene techo, pero introduce problemas nuevos: cómo dividir los datos, cómo coordinar las máquinas, cuánto tráfico de red se genera y qué hacer cuando una de ellas falla o se vuelve lenta.

Google formalizó una respuesta a estos problemas con el modelo MapReduce [1], que Hadoop popularizó como software libre junto con su sistema de archivos distribuido HDFS [2]. MapReduce resultó muy eficaz para trabajos de una sola pasada, pero ineficiente para los algoritmos iterativos típicos del aprendizaje automático, que recorren los mismos datos decenas de veces. Apache Spark [3], [4] nació precisamente para cubrir ese caso, manteniendo los datos en memoria entre iteraciones sin renunciar a la tolerancia a fallos.

Este trabajo tiene dos objetivos. El primero es explicar los conceptos centrales de Spark y de su biblioteca de aprendizaje automático, MLlib [6], tomando como caso de estudio el algoritmo de agrupamiento K-Means y comparándolo con el modelo relacional. El segundo es presentar una experiencia en dos partes: una introducción breve con tarjetas y una actividad web donde cada estudiante procesa, desde su celular, una partición de datos.

El resto del informe se organiza así: la Sección II presenta el marco teórico; la Sección III analiza casos de uso y compara Spark con el modelo relacional; la Sección IV analiza K-Means y su paralelización; la Sección V trata la tolerancia a fallos y el rendimiento; la Sección VI describe la experiencia de aula y sus resultados, y la Sección VII presenta las conclusiones.

II. Marco teórico

A. El modelo MapReduce

MapReduce [1] expresa un cómputo como dos funciones que escribe el programador. La función *map* se aplica en paralelo a cada fragmento de la entrada y emite pares clave–valor; la función *reduce* recibe todos los valores asociados a una misma clave y los combina. Entre ambas etapas, el sistema agrupa los pares por clave y los transfiere por la red (*shuffle*). Opcionalmente, una función *combiner* realiza una reducción parcial en cada nodo antes del *shuffle*, lo que reduce el volumen transmitido cuando la operación es asociativa y conmutativa.

El sistema se encarga del resto: divide la entrada, asigna tareas a las máquinas, reintenta las que fallan y lanza copias de respaldo de las tareas más lentas. Para garantizar la tolerancia a fallos, cada trabajo lee su entrada desde el sistema de archivos distribuido y escribe allí su salida, replicada en varios discos. Esta decisión es simple y robusta, pero tiene un costo: un algoritmo iterativo debe encadenar un trabajo por iteración y paga en cada uno la lectura y la escritura completas en disco.

B. Apache Spark y los RDD

Spark [3] se apoya en una abstracción llamada *Resilient Distributed Dataset* (RDD) [4]: una colección inmutable de registros, dividida en particiones repartidas entre las máquinas

del clúster. Sobre un RDD se definen dos clases de operaciones:

- **Transformaciones** (`map`, `filter`, `reduceByKey`, ...), que describen un RDD nuevo a partir de otro. Son *perezosas*: no se ejecutan al declararse, sino que se registran en un grafo dirigido acíclico (DAG) de dependencias.
- **Acciones** (`count`, `collect`, `save`, ...), que obligan a calcular un resultado y disparan la ejecución del DAG.

La evaluación perezosa permite al planificador ver el cómputo completo antes de ejecutarlo y fusionar en una misma etapa (*stage*) las transformaciones con dependencias estrechas, es decir, aquellas en las que cada partición de salida depende de una sola partición de entrada. Las dependencias amplias, como un `reduceByKey`, requieren un *shuffle* y marcan el límite entre etapas.

Dos propiedades distinguen a los RDD. La primera es la **persistencia**: el programador puede pedir con `cache()` o `persist()` que un RDD quede en la memoria de los nodos tras calcularse por primera vez, para reutilizarlo sin volver a leerlo del disco. La segunda es el **linaje** (*lineage*): en lugar de replicar los datos intermedios, cada RDD recuerda qué transformaciones lo produjeron a partir de datos estables. Si se pierde una partición, Spark la recalcula aplicando de nuevo esa receta solamente sobre la porción afectada [4].

Spark tampoco administra índices secundarios de propósito general como un motor relacional. Para reducir lecturas, Spark SQL puede aprovechar el plan de consultas, estadísticas y poda de particiones; los índices de un sistema de almacenamiento externo dependen de ese sistema y de su conector [13]. Del mismo modo, Spark no garantiza por sí solo transacciones ACID sobre cualquier destino: la atomicidad y consistencia de las escrituras dependen del almacenamiento y de su integración.

En la versión actual, la mayoría de los programas usan la API de *DataFrames* y Spark SQL, que agregan un esquema tabular y un optimizador de consultas sobre el mismo motor [5]. Los conceptos de partición, DAG, persistencia y linaje se mantienen.

C. Arquitectura de ejecución

Una aplicación Spark se compone de los siguientes roles [11], ilustrados en la Fig. 1:

- **Driver**: el proceso que ejecuta el programa principal. Construye el DAG, lo divide en etapas y tareas, envía las tareas a los *executors* y reúne los resultados.
- **Cluster manager** (o *master*): administra los recursos del clúster y asigna máquinas a cada aplicación. Puede ser el gestor *standalone* de Spark, YARN [7] o Kubernetes. No procesa datos.
- **Worker node**: cada máquina del clúster capaz de alojar procesos de la aplicación.
- **Executor**: proceso que corre en un *worker*, ejecuta tareas sobre particiones y conserva en su memoria los datos persistidos.

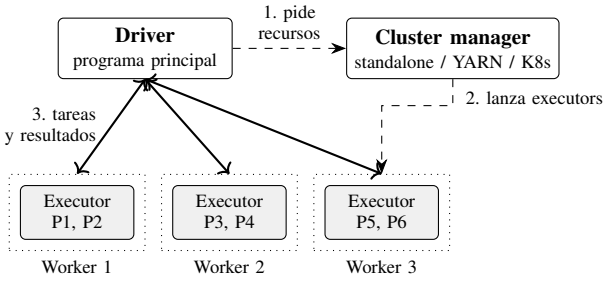


Fig. 1. Arquitectura de una aplicación Spark. Las líneas punteadas son tráfico de control; las continuas, el intercambio de tareas y resultados. Cada executor procesa sus particiones (P1–P6).

Una vez asignados los recursos, el *driver* se comunica directamente con los *executors*; el *cluster manager* no interviene en el procesamiento.

D. MLlib

MLlib es la biblioteca de aprendizaje automático de Spark [6]. Ofrece algoritmos de clasificación, regresión, agrupamiento, filtrado colaborativo y reducción de dimensionalidad, junto con herramientas para construir *pipelines* de preparación de datos. Su principal ventaja es que el mismo programa que se prueba sobre una muestra en una notebook se ejecuta sin cambios sobre miles de millones de registros en un clúster.

III. Casos de uso y comparación con el modelo relacional

A. Escenarios de aplicación

Spark resulta apropiado para trabajos analíticos de gran volumen que procesan muchos registros en conjunto. Un ejemplo es una tarea periódica de ETL que lee registros de ventas o eventos, limpia y transforma los datos y produce agregados para análisis con Spark SQL y DataFrames [13]. Otro es preparar características y entrenar modelos de agrupamiento, clasificación o recomendación con MLlib sobre conjuntos que se distribuyen entre varios nodos [12]. El K-Means del informe representa el caso de segmentación de clientes; en una aplicación real, los datos pueden provenir de transacciones ya almacenadas en archivos o en un lago de datos.

No conviene usar Spark como reemplazo de una base transaccional para operaciones pequeñas y frecuentes, como consultar o actualizar el saldo de una cuenta, validar restricciones y confirmar varias escrituras de manera atómica. Para esos casos, una base relacional ofrece transacciones y consultas puntuales; además, si los datos caben en una máquina, distribuirlos agrega costos de coordinación, serialización y red que pueden no compensar.

B. Diferencias y complementariedad

Spark no es una base NoSQL ni una base transaccional: es un motor distribuido de procesamiento. Spark SQL permite expresar operaciones sobre datos estructurados con SQL y DataFrames, pero el objetivo sigue siendo ejecutar análisis y

TABLA I
SPARK FRENTE A UNA BASE DE DATOS RELACIONAL TRADICIONAL

	Base relacional	**Apache Spark**
Objetivo	Consultas y transacciones operativas	Procesamiento analítico distribuido
Unidad de trabajo	Filas, tablas e índices	Particiones y tareas sobre conjuntos de datos
Acceso habitual	Lecturas y escrituras puntuales; transacciones	Lectura, transformación y agregación de lotes grandes
Escala	Servidor optimizado y réplicas según el motor	Clúster horizontal de *workers* y *executors*
Uso de SQL	Consultas transaccionales y analíticas	Spark SQL sobre datos distribuidos; no implica ser un motor OLTP

transformaciones distribuidas, no reemplazar el procesamiento transaccional de un motor relacional [13]. Las transacciones agrupan operaciones para que se apliquen completamente o no se apliquen, una propiedad central en los sistemas de bases de datos [15].

Spark tampoco replica automáticamente cada partición en memoria: el linaje permite reconstruir particiones perdidas a partir de sus transformaciones y de una fuente de datos disponible. La replicación y las garantías transaccionales dependen del sistema de almacenamiento de origen o destino y de su integración; no deben confundirse con una base de datos que administra transacciones ACID.

En una arquitectura habitual, ambos se complementan: la base relacional registra las operaciones del negocio y Spark lee esos datos para construir agregados o entrenar modelos. La diferencia relevante no es que uno sea siempre superior, sino el tipo de carga: transacciones puntuales frente a cómputo masivo por lotes.

IV. K-MEANS DISTRIBUIDO

A. El algoritmo

Dado un conjunto de n puntos $x_1, \ldots, x_n \in \mathbb{R}^d$ y un número k de grupos, K-Means busca k centroides $\mu_1, \ldots, \mu_k$ que minimicen la suma de los cuadrados de las distancias de cada punto a su centroide más cercano (WSSSE, *within-set sum of squared errors*):

$$J = \sum_{i=1}^{n} \min_{j} \|x_i - \mu_j\|^2. \tag{1}$$

El algoritmo de Lloyd [8], [9] aproxima una solución alternando dos pasos hasta que los centroides dejan de moverse:

1) **Asignación**: cada punto se asigna al centroide más cercano, $c(i) = \arg\min_j \|x_i - \mu_j\|^2$.
2) **Actualización**: cada centroide pasa a ser el promedio de los puntos que tiene asignados.

TABLA II
QUÉ ENVÍA CADA EXECUTOR AL DRIVER EN UNA ITERACIÓN

Opción	**Contenido**	**Tamaño**	**Correcto**
A	Todos los puntos	$\propto n$ (19 MiB para 10^6)	Sí
B	Promedio por grupo	48 B	No
C	Suma y conteo por grupo	72 B	Sí

Cada iteración no aumenta el costo J, por lo que el algoritmo converge, aunque puede hacerlo a un mínimo local que depende de los centroides iniciales.

B. Paralelización con estadísticos suficientes

El paso de asignación es trivialmente paralelo: cada *executor* solo necesita sus puntos y los k centroides actuales, que el *driver* le envía. El paso de actualización, en cambio, requiere combinar información de todas las particiones. Para ello, cada *executor* p calcula, por cada grupo j, la suma de las coordenadas y la cantidad de puntos asignados:

$$S_{p,j} = \sum_{i \in p,\, c(i)=j} x_i, \qquad N_{p,j} = \big|\{i \in p : c(i) = j\}\big|. \tag{2}$$

El *driver* suma estos valores y divide una sola vez:

$$\mu'_j = \frac{\sum_p S_{p,j}}{\sum_p N_{p,j}}. \tag{3}$$

El par (S,N) es un estadístico suficiente: contiene toda la información necesaria para calcular el promedio global y su combinación es asociativa y conmutativa, por lo que puede reducirse en árbol o en cualquier orden. El tamaño del mensaje es $k(d+1)$ números por partición, independiente de la cantidad de puntos. MLlib implementa así cada iteración: una agregación de sumas y conteos por grupo sobre el RDD persistido [6].

C. El error del promedio de promedios

Una alternativa tentadora es que cada *executor* envíe directamente el promedio de sus puntos por grupo, $\bar{x}_{p,j} = S_{p,j}/N_{p,j}$, y que el *driver* promedie esos promedios. El resultado es incorrecto siempre que las particiones tengan distinta cantidad de puntos en un mismo grupo, porque se pierde el peso $N_{p,j}$. Por ejemplo, si una partición tiene un único punto en $x = 10$ y otra tiene tres puntos en $x = 0$:

$$\underbrace{\frac{10+0}{2} = 5}_{\text{promedio de promedios}} \quad \neq \quad \underbrace{\frac{10+0+0+0}{4} = 2{,}5}_{\text{promedio real}}. \tag{4}$$

La Tabla II compara las tres estrategias que la actividad propone a los estudiantes, con $k = 3$, $d = 2$ y valores de 8 bytes.

La opción A es correcta pero traslada todos los datos al *driver*, lo que anula la ventaja de distribuir y convierte al *driver* en un cuello de botella. La opción B es pequeña pero incorrecta. La opción C es pequeña y correcta: es el patrón de *combiner* de MapReduce aplicado a K-Means.

D. Inicialización y parámetros en MLlib

Como el resultado depende de los centroides iniciales, MLlib usa por defecto la inicialización *k-means*‖ [10], una variante paralelizable de *k-means++* que elige centroides iniciales dispersos en pocas pasadas sobre los datos. Los parámetros principales del estimador `pyspark.ml.clustering.KMeans` son la cantidad de grupos `k`, el máximo de iteraciones `maxIter` (20 por defecto), la tolerancia de convergencia `tol` y la semilla `seed` [12]. El Listado 1 muestra un programa completo.

Listado 1. K-Means con PySpark y MLlib

```
from pyspark.ml.clustering import KMeans
from pyspark.ml.feature import VectorAssembler

df = spark.read.csv("clientes.csv",
                    header=True, inferSchema=True)
vec = VectorAssembler(inputCols=["x", "y"],
                      outputCol="features")
datos = vec.transform(df).cache()

modelo = KMeans(k=3, maxIter=12, seed=1).fit(datos)
print(modelo.clusterCenters())
print(modelo.summary.trainingCost)  # WSSSE
```

La llamada a `cache()` es la que convierte a Spark en una herramienta adecuada para este algoritmo: sin ella, cada iteración volvería a leer y a convertir el archivo de entrada.

V. Tolerancia a fallos y rendimiento

A. Recuperación por linaje

En un clúster de cientos de máquinas, las fallas son eventos normales. MapReduce y HDFS las toleran replicando los datos en disco; Spark, en cambio, reconstruye las particiones perdidas a partir de su linaje [4]. Si un *executor* se cae durante una iteración de K-Means, el *driver* detecta que sus tareas no terminaron y las reasigna a otros *executors*, que vuelven a leer la porción correspondiente de la entrada, reaplican las transformaciones y continúan. Solo se recalcula lo perdido, sin replicar en memoria cada resultado intermedio.

B. Rezagados y ejecución especulativa

Como cada iteración termina cuando termina su tarea más lenta, una sola máquina rezagada (*straggler*), por un disco degradado o por competir con otros procesos, retrasa a todo el clúster. Dean y Ghemawat propusieron lanzar *tareas de respaldo*: cuando el trabajo está por terminar, las tareas que siguen en curso se copian en otras máquinas y se usa el resultado de la que termine primero [1]. Spark implementa la misma idea con la opción `spark.speculation`, desactivada por defecto. Al activarla, una tarea se relanza cuando dura más que un múltiplo (`spark.speculation.multiplier`, 3 en la documentación actual) de la mediana de las tareas de su etapa, una vez completada una fracción de ellas (`spark.speculation.quantile`, 0,9) [14].

TABLA III
Hadoop MapReduce y Apache Spark

	Hadoop MapReduce	**Spark**
Datos entre pasos	Disco (HDFS)	Memoria (`cache`)
Modelo	Map y reduce fijos	DAG de transformaciones
Tolerancia a fallos	Replicación en disco	Linaje y recálculo
Iterativo	Un trabajo por iteración	Un programa, datos reutilizados
APIs	Java	Scala, Java, Python, R, SQL

C. Spark frente a Hadoop en algoritmos iterativos

Sea r el tiempo de leer una partición desde el disco, c el de procesarla y w el de escribir el resultado intermedio. Para m iteraciones, un encadenamiento de trabajos MapReduce cuesta aproximadamente

$$T_{\text{Hadoop}} \approx m\,(r + c + w), \tag{5}$$

mientras que Spark, con los datos persistidos en memoria, lee una sola vez:

$$T_{\text{Spark}} \approx r + m\,c. \tag{6}$$

Cuando la lectura domina ($r \gg c$), la ventaja crece casi linealmente con la cantidad de iteraciones. Zaharia et al. reportaron mejoras de hasta 20 veces en algoritmos iterativos de aprendizaje automático [4]. La Tabla III resume las diferencias.

Spark no es siempre la mejor opción. Si los datos caben en la memoria de una sola máquina, bibliotecas locales como pandas o scikit-learn suelen ser más rápidas, porque evitan el costo de coordinar procesos, serializar datos y transmitirlos por la red. El procesamiento distribuido se justifica cuando el volumen o el tiempo de cómputo superan lo que una máquina puede resolver.

VI. Experiencia presencial y actividad web

A. Objetivo y diseño

La exposición completa está planificada para 20 minutos: cinco para la práctica presencial, dos para registrar e interpretar sus resultados, tres para la presentación teórica, siete para la actividad web y tres para el cierre. En la primera ronda presencial, siete participantes clasifican las mismas 200 fichas numeradas del 1 al 200 como pares o impares, trabajando alrededor de un único montón compartido. En la segunda, las fichas se reparten en cuatro particiones de 50, procesadas por cuatro mesas de cinco personas. Un expositor coordina el inicio y reúne los resultados como analogía del *driver*; otros cuatro registran el tiempo y las fichas mal clasificadas por mesa. El tiempo paralelo de la segunda ronda es el de la última mesa en terminar, no la suma de los tiempos parciales.

La pantalla de carga registra la duración de la ronda tradicional, la duración de cada mesa distribuida y el número de fichas mal clasificadas en cada caso. La diapositiva de resultados muestra el tiempo de la última mesa distribuida,

TABLA IV
FASES DE LA EXPERIENCIA WEB

Fase	Qué hacen los estudiantes	Concepto
Unirse	Escanean el QR y reciben su partición	Driver, executor, partición
Procesar partición	Inician el cálculo de sus puntos; la web los asigna al centroide más cercano	Partición, tarea, asignación local
Enviar	Eligen enviar todos los puntos (A) o sumas y cantidades (C)	Tráfico de red y agregación parcial
Reduce	El *driver* combina resultados y mueve los centroides	Reducción y una iteración de K-Means

suma sus errores y calcula la diferencia de tiempo entre rondas. En la primera ronda, el conjunto permanece en un único montón compartido; en la segunda, cada grupo recibe una partición independiente para trabajar en paralelo. Esta comparación es didáctica, no un benchmark controlado ni una medición de Apache Spark: la cantidad de participantes y la organización cambian entre rondas, y el trabajo humano no es equivalente al cómputo de máquinas. Por ello, los tiempos solo permiten comentar esta experiencia y no atribuir a Spark una aceleración medida.

La experiencia tecnológica principal es una actividad web de aproximadamente siete minutos para 27 participantes. El proyector muestra la consola del *driver*; cada estudiante se une con un QR y el celular representa un *executor*, con una partición de 8 puntos de un conjunto sintético de clientes y tres centroides. Cada estudiante asigna manualmente un punto al centroide que considera más cercano; la aplicación confirma la decisión y asigna automáticamente los otros siete para mantener breve la actividad. Para comparar la agregación local, la mitad izquierda del aula envía todos los puntos (A) y la mitad derecha envía sumas y cantidades (C). El *driver* combina ambos tipos de resultados y actualiza los centroides. Se omite la alternativa del promedio parcial (B) en el recorrido principal para reducir la carga cognitiva; se conserva en el análisis teórico del informe.

El sistema está implementado en Node.js con Express y Socket.io. Un servidor central mantiene el estado autoritativo de la simulación (particiones, centroides, tareas en curso) y lo sincroniza con el proyector y con los celulares mediante WebSockets. Puede servirse desde una computadora personal en la red del aula, sin servicios de terceros para almacenar datos. Un modo de alto contraste permite usarlo con el aula iluminada.

La aplicación no ejecuta Apache Spark ni convierte los celulares en *executors* Spark reales: representa didácticamente los roles, las particiones y el flujo de agregación para que la audiencia los experimente.

B. Fases del recorrido principal

La Tabla IV resume las fases de la experiencia web y el concepto que ilustra cada una.

La secuencia oral conecta las dos rondas con los términos partición, paralelismo y agregación; después explica el *driver*, los *workers*/*executors*, la evaluación perezosa, K-Means y MLlib. Para reservar tiempo a la participación, las fases automáticas, la simulación de fallos, la comparación de tiempos con Hadoop y la vista de código quedan como material opcional, fuera del recorrido de 20 minutos.

C. Registro de observaciones

La pantalla de carga presencial registra segundos y cantidad de fichas mal clasificadas: una medición para la ronda tradicional y una para cada una de las cuatro mesas distribuidas. La presentación calcula el tiempo de la última mesa y la suma de errores de las cuatro particiones. Los datos se introducen durante la exposición y describen el desempeño de los participantes en el aula; no son una medición del rendimiento de Spark.

En la ronda web, la consola muestra cuántos participantes enviaron resultados y cuánto tráfico simulado se generó al enviar puntos completos o resúmenes. El volumen en bytes representa el tamaño codificado de los mensajes definido por la aplicación, no una medición del tráfico de red de un clúster Spark real.

D. Limitaciones

La actividad simplifica deliberadamente varios aspectos. No representa al *cluster manager*, porque los celulares ya están conectados al iniciar; cada celular cumple a la vez el papel de *worker* y de *executor*, y los datos son bidimensionales para poder dibujarlos. El servidor central es un punto único de falla, igual que el *driver* en Spark: si se detiene, la actividad debe reiniciarse.

Si la red no está disponible, se reemplaza la ronda web por tarjetas con coordenadas y tres centroides impresos: las mesas asignan cada punto al centro más cercano, envían sumas y cantidades, y el grupo expositor calcula los nuevos centroides.

VII. CONCLUSIONES

Apache Spark reduce el costo de los algoritmos iterativos al permitir reutilizar datos en memoria y reconstruir particiones perdidas mediante linaje. K-Means muestra cómo una tarea puede repartirse: cada *executor* asigna los puntos de su partición y calcula resúmenes; luego, el *driver* combina sumas y cantidades para mover los centroides. La experiencia presencial compara la clasificación manual de un montón compartido de 200 fichas con la de cuatro particiones de 50 en paralelo; registra tiempos y errores observados, y explicita las limitaciones de esa comparación. Luego, en una ronda web de K-Means, la audiencia participa como *executors* simulados. En conjunto, ambas prácticas conectan la intuición de repartir trabajo con las decisiones concretas de Spark y MLlib.

REFERENCIAS

[1] J. Dean and S. Ghemawat, “MapReduce: Simplified data processing on large clusters,” in *Proc. 6th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, San Francisco, CA, USA, 2004, pp. 137–150.

[2] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The Hadoop distributed file system," in *Proc. IEEE 26th Symp. Mass Storage Systems and Technologies (MSST)*, 2010, pp. 1–10.
[3] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, "Spark: Cluster computing with working sets," in *Proc. 2nd USENIX Workshop Hot Topics in Cloud Computing (HotCloud)*, 2010.
[4] M. Zaharia *et al.*, "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proc. 9th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, San Jose, CA, USA, 2012, pp. 15–28.
[5] M. Zaharia *et al.*, "Apache Spark: A unified engine for big data processing," *Commun. ACM*, vol. 59, no. 11, pp. 56–65, 2016.
[6] X. Meng *et al.*, "MLlib: Machine learning in Apache Spark," *J. Mach. Learn. Res.*, vol. 17, no. 34, pp. 1–7, 2016.
[7] V. K. Vavilapalli *et al.*, "Apache Hadoop YARN: Yet another resource negotiator," in *Proc. 4th ACM Symp. Cloud Computing (SoCC)*, 2013, pp. 5:1–5:16.
[8] S. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Inf. Theory*, vol. 28, no. 2, pp. 129–137, 1982.
[9] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symp. Mathematical Statistics and Probability*, vol. 1, 1967, pp. 281–297.
[10] B. Bahmani, B. Moseley, A. Vattani, R. Kumar, and S. Vassilvitskii, "Scalable k-means++," *Proc. VLDB Endowment*, vol. 5, no. 7, pp. 622–633, 2012.
[11] The Apache Software Foundation, "Cluster mode overview," *Apache Spark Documentation*. [Online]. Available: https://spark.apache.org/docs/latest/cluster-overview.html (accedido: nov. 2026).
[12] The Apache Software Foundation, "Clustering — K-means," *Apache Spark MLlib Guide*. [Online]. Available: https://spark.apache.org/docs/latest/ml-clustering.html (accedido: nov. 2026).
[13] The Apache Software Foundation, "Spark SQL, DataFrames and Datasets Guide," *Apache Spark Documentation*. [Online]. Available: https://spark.apache.org/docs/latest/sql-programming-guide.html (accedido: nov. 2026).
[14] The Apache Software Foundation, "Spark configuration," *Apache Spark Documentation*. [Online]. Available: https://spark.apache.org/docs/latest/configuration.html (accedido: nov. 2026).
[15] The PostgreSQL Global Development Group, "Transactions," *PostgreSQL Documentation*. [Online]. Available: https://www.postgresql.org/docs/current/tutorial-transactions.html (accedido: nov. 2026).